

A Practical Guide to Hyperparameter optimization

TOPIC -- HYPERPARAMETER OPTIMIZATION

-- 01 --

Bayesian optimization is a global optimization method for noisy black-box functions. Applied to hyperparameter optimization, Bayesian optimization builds a probabilistic model of the function mapping from hyperparameter values to the objective evaluated on a validation set. By iteratively evaluating a promising hyperparameter configuration based on the current model, and then updating it, Bayesian optimization aims to gather observations revealing as much information as possible about this function and, in particular, the location of the optimum. It tries to balance exploration (hyperparameters for which the outcome is most uncertain) and exploitation (hyperparameters expected close to the optimum). In practice, Bayesian optimization has been shown to obtain better results in fewer evaluations compared to grid search and random search, due to the ability to reason about the quality of experiments before they are run.

-- 02 --

Evolutionary optimization is a methodology for the global optimization of noisy black-box functions. In hyperparameter optimization, evolutionary optimization uses evolutionary algorithms to search the space of hyperparameters for a given algorithm. Evolutionary hyperparameter optimization follows a process inspired by the biological concept of evolution:

-- 03 --

In machine learning, hyperparameter optimization or tuning is the problem of choosing a set of optimal hyperparameters for a learning algorithm. A hyperparameter is a parameter whose value is used to control the learning process, which must be configured before the process starts. Hyperparameter optimization determines the set of hyperparameters that yields an optimal model which minimizes a predefined loss function on a given data set. The objective function takes a set of hyperparameters and returns the associated loss. Cross-validation is often used to estimate this generalization performance, and therefore choose the set of values for hyperparameters that maximize it.

-- 04 --

Population-based Population Based Training (PBT) learns both hyperparameter values and network weights. Multiple learning processes operate independently,

using different hyperparameters. As with evolutionary methods, poorly performing models are iteratively replaced with models that adopt modified hyperparameter values and weights based on the better performers. This replacement model warm starting is the primary differentiator between PBT and other evolutionary methods. PBT thus allows the hyperparameters to evolve and eliminates the need for manual hypertuning. The process makes no assumptions regarding model architecture, loss functions or training procedures. PBT and its variants are adaptive methods: they update hyperparameters during the training of the models. On the contrary, non-adaptive methods have the sub-optimal strategy to assign a constant set of hyperparameters for the whole training.

-- 05 --

Issues with hyperparameter optimization When hyperparameter optimization is done, the set of hyperparameters are often fitted on a training set and selected based on the generalization performance, or score, of a validation set. However, this procedure is at risk of overfitting the hyperparameters to the validation set. Therefore, the generalization performance score of the validation set (which can be several sets in the case of a cross-validation procedure) cannot be used to simultaneously estimate the generalization performance of the final model. In order to do so, the generalization performance has to be evaluated on a set independent (which has no intersection) of the set (or sets) used for the optimization of the hyperparameters, otherwise the performance might give a value which is too optimistic (too large). This can be done on a second test set, or through an outer cross-validation procedure called nested cross-validation, which allows an unbiased estimation of the generalization performance of the model, taking into account the bias due to the hyperparameter optimization.

-- 06 --

Random search Random Search replaces the exhaustive enumeration of all combinations by selecting them randomly. This can be simply applied to the discrete setting described above, but also generalizes to continuous and mixed spaces. A benefit over grid search is that random search can explore many more values than grid search could for continuous hyperparameters. It can outperform Grid search, especially when only a small number of hyperparameters affects the final performance of the machine learning algorithm. In this case, the optimization problem is said to have a low intrinsic dimensionality. Random Search is also embarrassingly parallel, and additionally allows the inclusion of prior knowledge by specifying the distribution from which to sample. Despite its simplicity, random search remains one of the important base-lines against which to compare the performance of new hyperparameter optimization methods.

-- 07 --

Gradient-based optimization For specific learning algorithms, it is possible to compute the gradient with respect to hyperparameters and then optimize the hyperparameters using gradient descent. The first usage of these techniques was focused on neural networks. Since then, these methods have been extended to other models such as support vector machines or logistic regression. A different approach in order to obtain a gradient with respect to hyperparameters consists in differentiating the steps of an iterative optimization algorithm using automatic differentiation. A more recent work along this direction uses the implicit function theorem to calculate hypergradients and proposes a stable approximation of the inverse Hessian. The method scales to millions of hyperparameters and requires constant memory. In a different approach, a hypernetwork is trained to approximate the best response function. One of the advantages of this method is that it can handle discrete hyperparameters as well. Self-tuning networks offer a memory efficient version of this approach by choosing a compact representation for the hypernetwork. More recently, Δ -STN has improved this method further by a slight reparameterization of the hypernetwork which speeds up training. Δ -STN also yields a better approximation of the best-response Jacobian by linearizing the network in the weights, hence removing unnecessary nonlinear effects of large changes in the weights. Apart from hypernetwork approaches, gradient-based methods can be used to optimize discrete hyperparameters also by adopting a continuous relaxation of the parameters. Such methods have been extensively used for the optimization of architecture hyperparameters in neural architecture search.

-- 08 --

Grid search The traditional method for hyperparameter optimization has been grid search, or a parameter sweep, which is simply an exhaustive searching through a manually specified subset of the hyperparameter space of a learning algorithm. A grid search algorithm must be guided by some performance metric, typically measured by cross-validation on the training set or evaluation on a hold-out validation set. Since the parameter space of a machine learner may include real-valued or unbounded value spaces for certain parameters, manually set bounds and discretization may be necessary before applying grid search. For example, a typical soft-margin SVM classifier equipped with an RBF kernel has at least two hyperparameters that need to be tuned for good performance on unseen data: a regularization constant C and a kernel hyperparameter γ . Both parameters are continuous, so to perform grid search, one selects a finite set of "reasonable" values for each, say